

Day 3 – Principles of Multi-Objective Optimisation

Dr Lauren Ye Seol LEE

Lecturer in Data-driven Chemical PSE,
Department of Chemical Engineering,
University College London (UCL)

lauren.lee@ucl.ac.uk



Learning Objectives:

- **Formulate:** a multi-objective optimisation problem and explain why conflicting objectives arise in engineering applications.
- **Identify and interpret:** dominated, non-dominated, and Pareto-optimal solutions in both decision space and objective space.
- **Compare:** key approaches for generating trade-off solutions e.g., scalarisation methods and population-based evolutionary methods.
- **Assess:** the quality of an approximated Pareto front in terms of feasibility, convergence, coverage, distribution, and hypervolume.

My Trajectory



Dr Lauren Ye Seol LEE

Assistant Professor (Lecturer) in Data-Driven Chemical PSE

- 2023: Joined UCL
- 2017–2023: Dept. Chemical Engineering of ICL (acquired PhD in 2022)
- 2013–2017: Hanwha Ocean (Researcher in Energy R&D Team)



Research Focus:

Computer-aided molecular design, molecular optimisation, and data-driven process systems engineering.

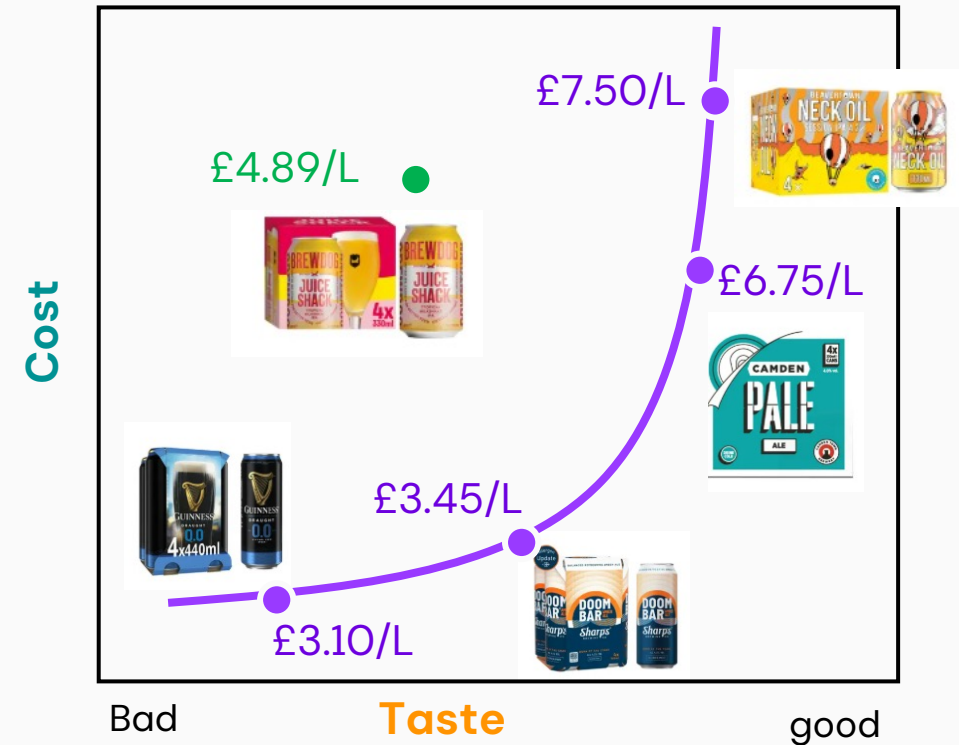
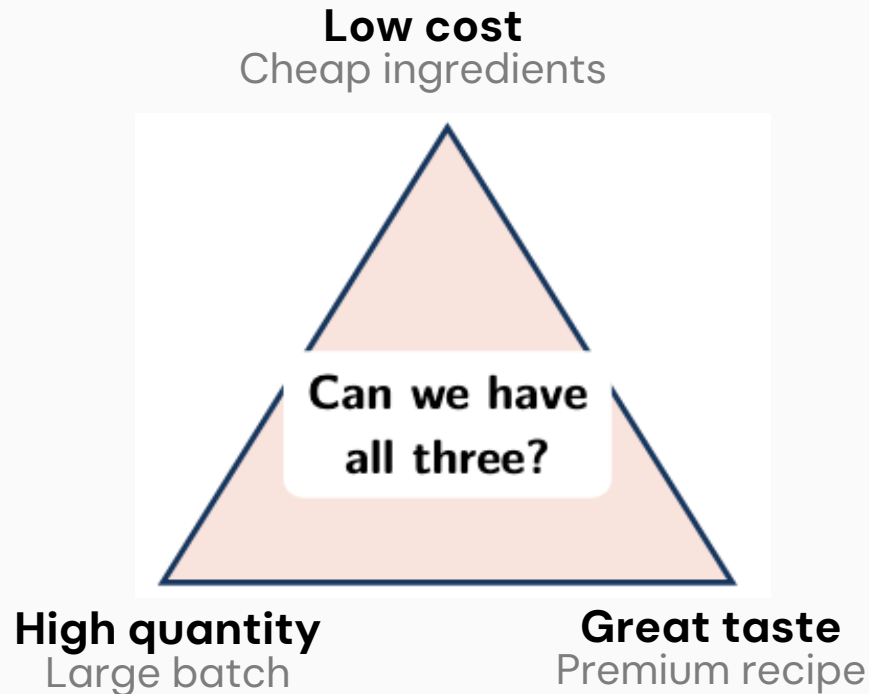
My research develops AI-enabled design and optimisation frameworks that integrate molecular modelling, mechanistically grounded machine learning, optimisation, and automated experimentation to accelerate molecular discovery and decision-making across multiple scales.

Why Multi-objective Optimisation (MOO)?

Most real-world problems are multi-objective!

○ Beer challenge:

- You have one recipe, a limited budget, and thirsty guests. What should you optimise?



Usually, improving one objective means accepting a loss somewhere else.

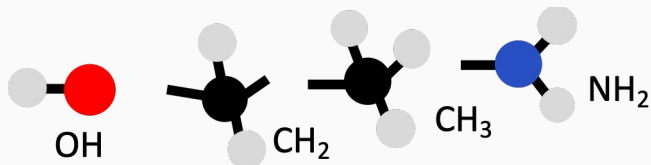
Why Multi-objective Optimisation (MOO)?

The same structure appears across engineering and design.

It involves more than one objective function related to performance, economics, safety and reliability, which have to be optimised simultaneously since these objective functions may be fully or partially conflicting over the range of interest.

Molecular Design

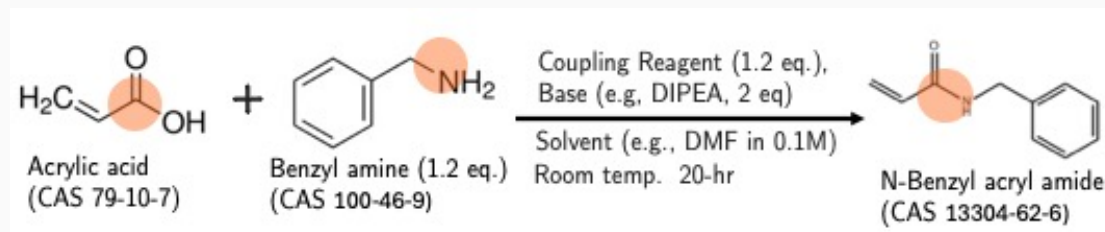
Solvent Design for CO₂ chemical absorption



- CO₂ solubility (=CO₂ loading) ↑
- Density ↑
- Heat capacity ↓
- Vapor pressure ↓

Reaction Condition Optimisation

Choice of synthesis condition: reagent, base, solvent, reaction time..



- Yield of target ↑
- Side product ↓
- Cost of substrate ↓

Generic Mathematical Formulation

The mathematical form changes little

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{X}} \quad & \mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})) \\ \text{subject to} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, p, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, q, \\ & \mathbf{x}^L \leq \mathbf{x} \leq \mathbf{x}^U. \end{aligned}$$

Decision space

Recipe settings, molecular representation, operating conditions, or any other controllable decisions.

Objective space

The resulting cost, yield, taste, energy, performance, and environmental measures.

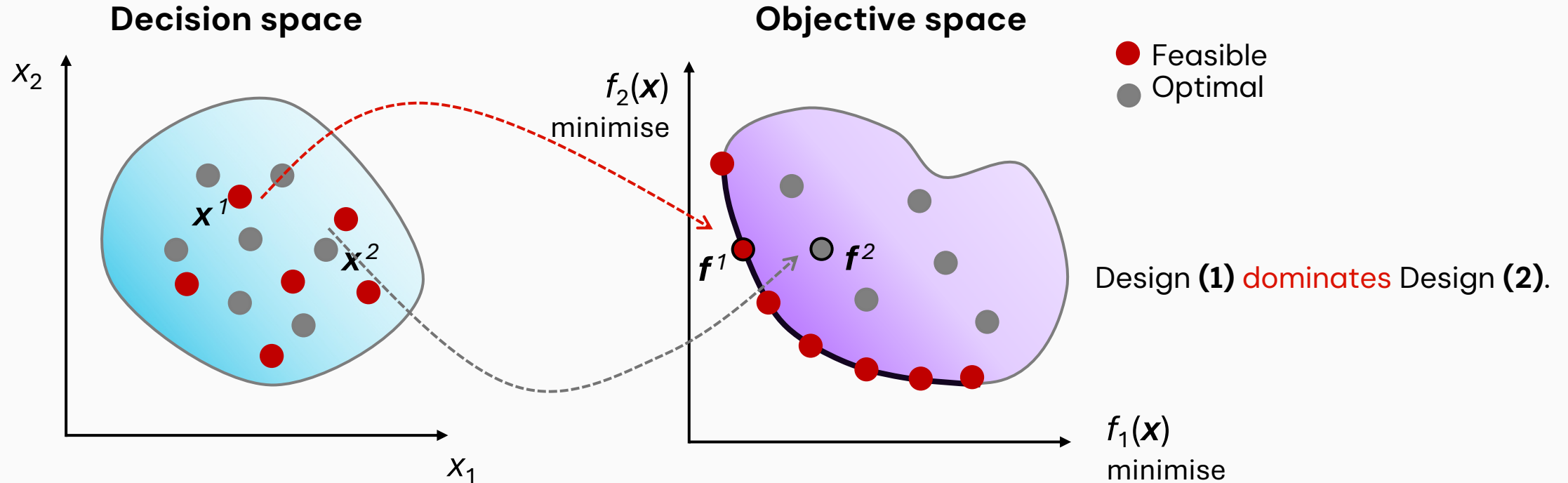
Key difference from SOO

There is generally no single minimiser. The answer is a set of defensible alternatives.

If objective values are vectors, what does it mean for optimal solutions??!!!

Dominance: Pareto-optimality

In MOO, the **goodness** of a solution is determined by the **dominance**.

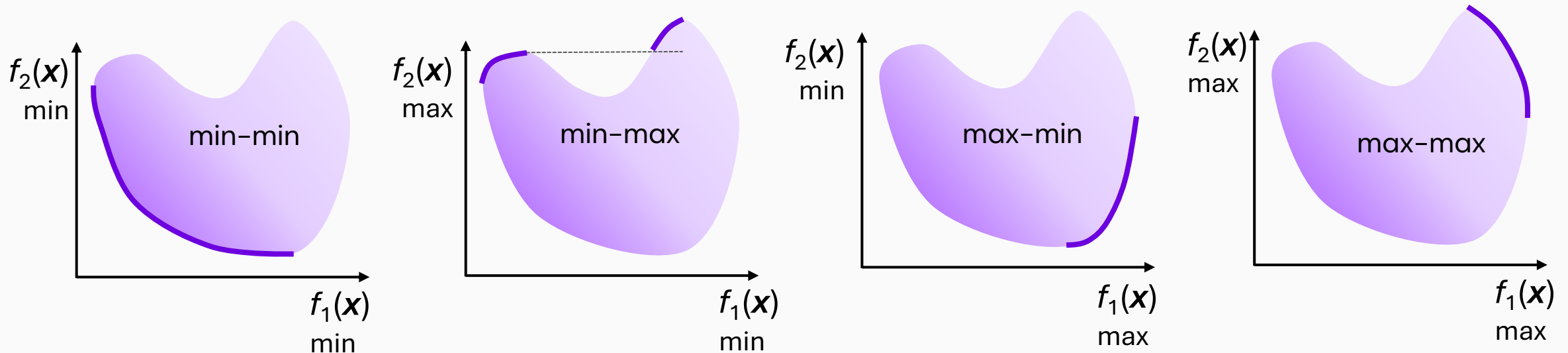


- x^1 **dominate** x^2 , if:
 - ✓ Solution x^1 is **no worse** than x^2 in **all** objectives.
 - ✓ Solution x^1 is **strictly better** than x^2 in **at least one** objective.
- If x^1 **dominates** x^2 , then x^2 is **dominated by** x^1
- A set of **non-dominated solutions** → **Pareto-optimal set**
→ their objective vector **form** **Pareto front**

Dominance: Pareto-optimality

Examples

The position of the Pareto front depends on whether each objective is minimized or maximized.



The direction changes, but the principle remains the same:

At a Pareto-optimal point, **improving** one objective requires **sacrificing at least** one other objective.

Dominance: Pareto-optimality

○ Pareto dominance

Consider two feasible solutions $\mathbf{x}^{(a)}$ and $\mathbf{x}^{(b)}$.

We say that $\mathbf{x}^{(a)}$ **dominates** $\mathbf{x}^{(b)}$ if both of the following conditions hold:

1. $\mathbf{x}^{(a)}$ is no worse than $\mathbf{x}^{(b)}$ in every objective: $f_k(\mathbf{x}^{(a)}) \leq f_k(\mathbf{x}^{(b)}), \quad \forall k \in \{1, \dots, M\}.$
2. $\mathbf{x}^{(a)}$ is strictly better than $\mathbf{x}^{(b)}$ in at least one objective: $f_k(\mathbf{x}^{(a)}) < f_k(\mathbf{x}^{(b)}),$

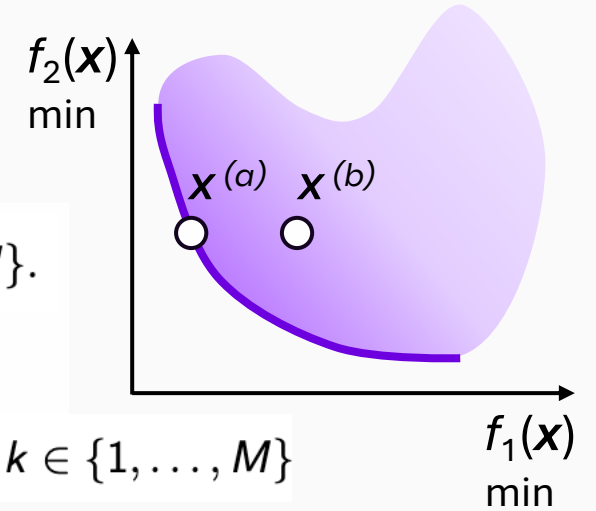
Mathematically, this is written as:

$$\mathbf{x}^{(a)} \prec \mathbf{x}^{(b)},$$

Or equivalently

$$\mathbf{f}(\mathbf{x}^{(a)}) \preceq \mathbf{f}(\mathbf{x}^{(b)}) \quad \text{and} \quad \mathbf{f}(\mathbf{x}^{(a)}) \neq \mathbf{f}(\mathbf{x}^{(b)}).$$

for at least one $k \in \{1, \dots, M\}$



○ Pareto optimality and non-dominated solutions

Let $\mathcal{X}$ denote the feasible decision space.

A feasible solution $\mathbf{x}^* \in \mathcal{X}$ is called **Pareto-optimal** if there exists no other feasible solution $\mathbf{x}$ that dominates it"

$$\nexists \mathbf{x} \in \mathcal{X} \quad \text{such that} \quad \mathbf{x} \prec \mathbf{x}^*.$$

The set of all **Pareto-optimal solutions** in **decision space** is called the **Pareto set**"

$$\mathcal{P}^* = \{\mathbf{x}^* \in \mathcal{X} \mid \nexists \mathbf{x} \in \mathcal{X} \text{ such that } \mathbf{x} \prec \mathbf{x}^*\}.$$

Their corresponding objective space is called the **Pareto front (=Pareto points)**:

$$\Psi^* = \{\mathbf{f}(\mathbf{x}) \mid \mathbf{x} \in \mathcal{P}^*\}$$



Some Others..

Ideal | Utopia | Nadir

○ Ideal objective vector:

The m^{th} component of the **ideal objective vector** $\mathbf{z}^*$ is obtained by independently solving the constrained single-objective problem

$$\begin{aligned} f_k^* &= \min_{\mathbf{x}} f_k(\mathbf{x}) \\ \text{subject to } & \mathbf{x} \in \mathcal{X}, \end{aligned} \quad k = 1, \dots, m.$$

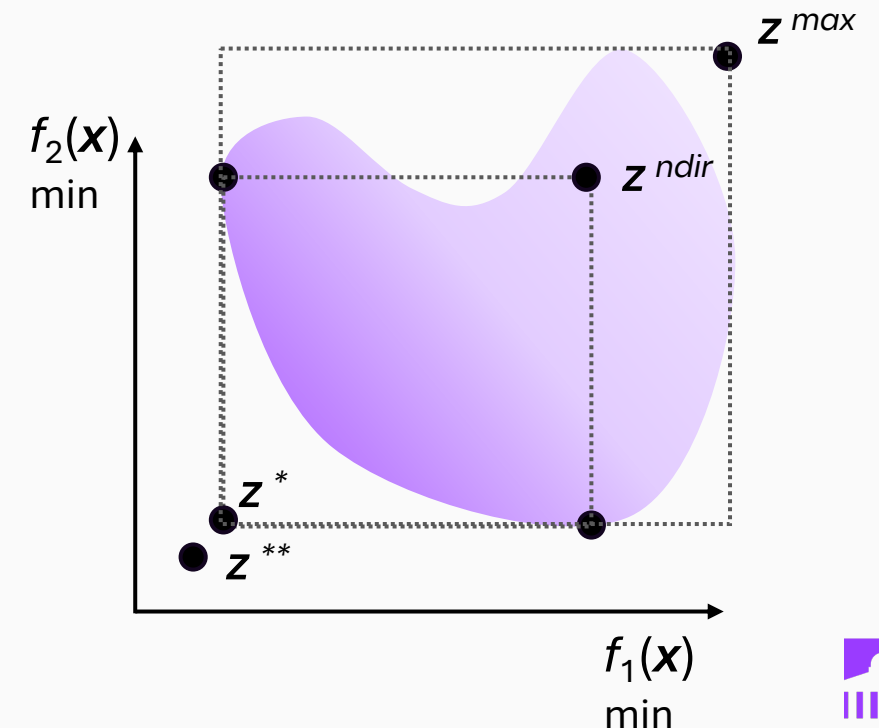
$$\mathbf{z}^* = \mathbf{f}^* = (f_1^*, f_2^*, \dots, f_m^*)^T$$

○ Utopian point

A utopian objective vector $\mathbf{z}^{**}$ has each of its components marginally smaller than that of the ideal objective vector, or $\mathbf{z}_i^{**} = \mathbf{z}_i^* - \varepsilon_i$ with $\varepsilon_i > 0$ for all $i = 1, 2, \dots, m$

○ Nadir point

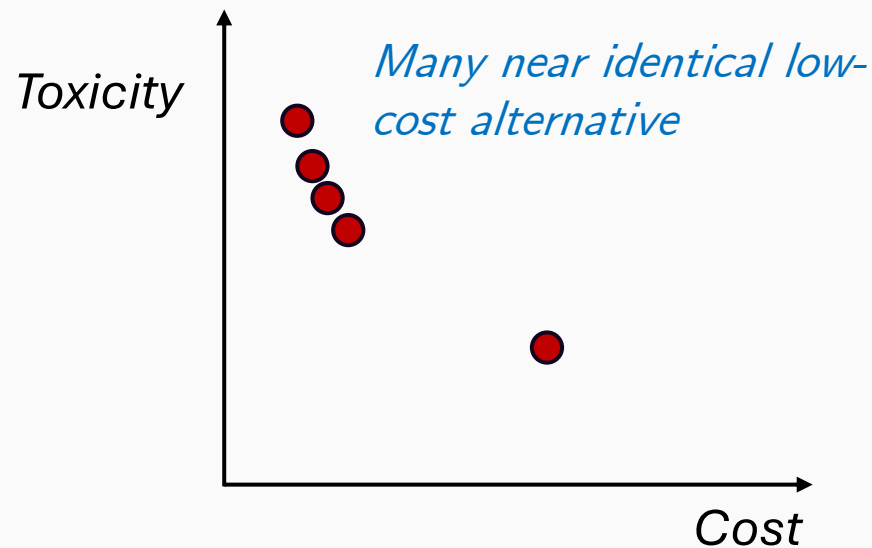
A nadir objective vector $\mathbf{z}^{\text{nadir}}$ represents the upper bound of each objective in the entire Pareto-optimal set, and not in the entire search space.



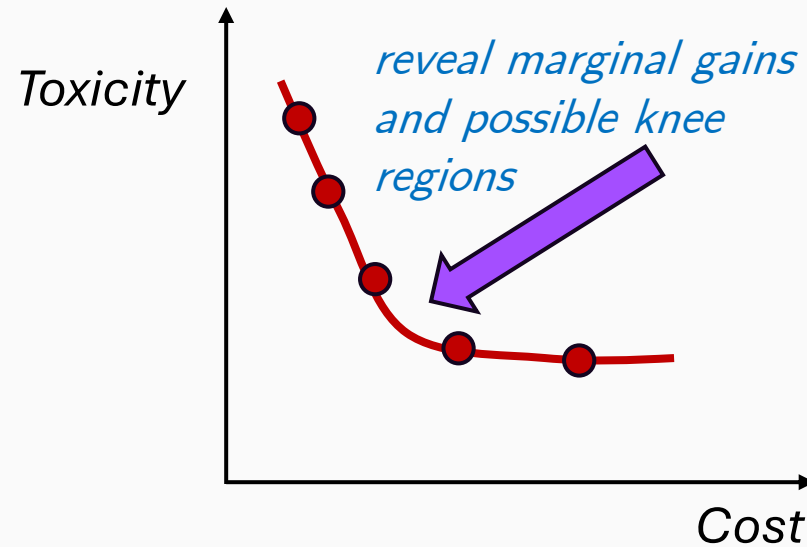
Principles of MOO

Goal of MOO

- Find a set of solutions as **close** as possible to Pareto-optimal front.
- Find a set of solutions as **diverse** as possible.
- ✓ Especially in an **a posteriori approach**, preferences are articulated after seeing the attainable trade-offs.
- ✓ A good approximation must therefore support comparison, not merely satisfy non-dominance.



Non-dominated, but not decision-useful



Representative trade-off set

Scalarisation method

- Transform the multiple objectives into a sequence of SOO problems.
 - Different parameter values then produce different trade-off solutions.
-
- Weighted sum method
 - ϵ -Constraint method
 - Weighted metric method
 - Rotated weighted metric method
 - (modified) Normal boundary intersection method
 - ...

Weighted Sum Method

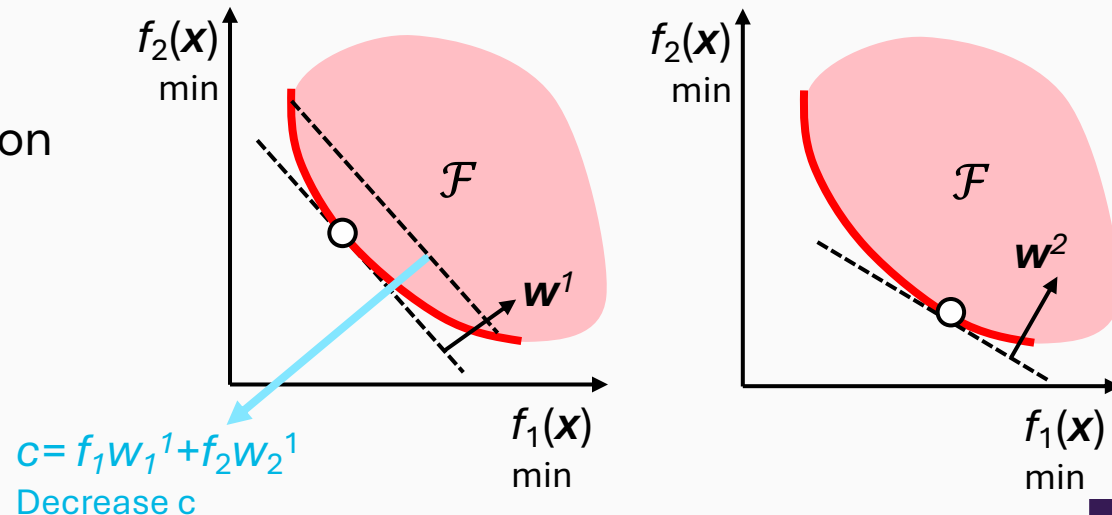
Weighted sum turn many objectives into one familiar problem

$$\min_{\mathbf{x} \in \mathcal{X}} \quad \sum_{i=1}^m w_i \tilde{f}_i(\mathbf{x}), \quad w_i \geq 0, \quad \sum_i w_i = 1 \quad \tilde{f}_k(\mathbf{x}) = \frac{f_k(\mathbf{x}) - f_k^{\text{ndir}}(\mathbf{x})}{f_k^{\text{ideal}}(\mathbf{x}) - f_k^{\text{ndir}}(\mathbf{x})}, \quad k = 1, \dots, m.$$

Scalarize a set of objectives into **a single objective**
by adding each objective multiplied by a **user-specified weight**
(=relative importance)

Why use it?

- Simple and intuitive.
- Can use established (single) optimisation solvers.
- No additional constraints involved.
- Note: should normalize the objective function.



Weighted Sum Method

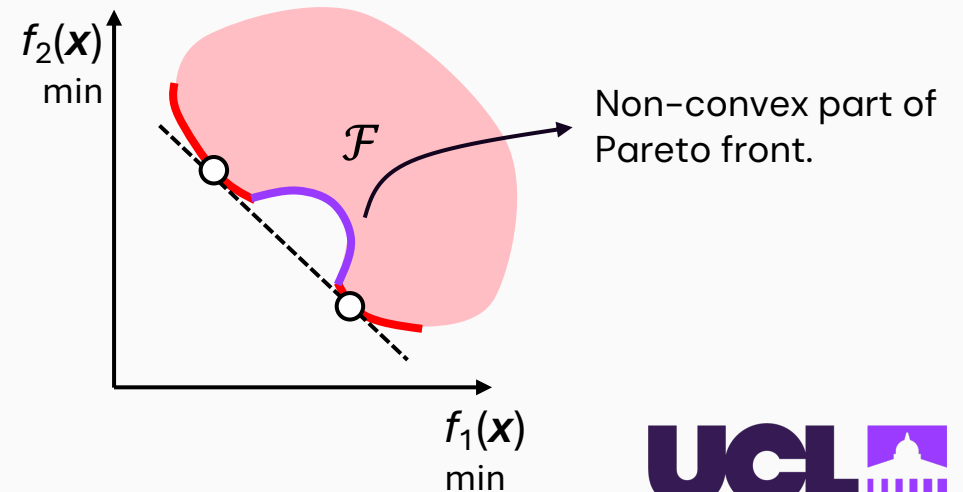
Weighted sum turn many objectives into one familiar problem

$$\min_{\mathbf{x} \in \mathcal{X}} \quad \sum_{i=1}^m w_i \tilde{f}_i(\mathbf{x}), \quad w_i \geq 0, \quad \sum_i w_i = 1$$

Scalarize a set of objectives into **a single objective**
by adding each objective multiplied by a **user-specified weight**
(=relative importance)

Disadvantages

- For a **non-convex front**, non-dominated points are **not supported**.
- Uniformly distributed set of weights **does not guarantee** a uniformly distributed set of Pareto points.
- Two different set of weights **not necessarily lead to** two different Pareto optimal solutions.

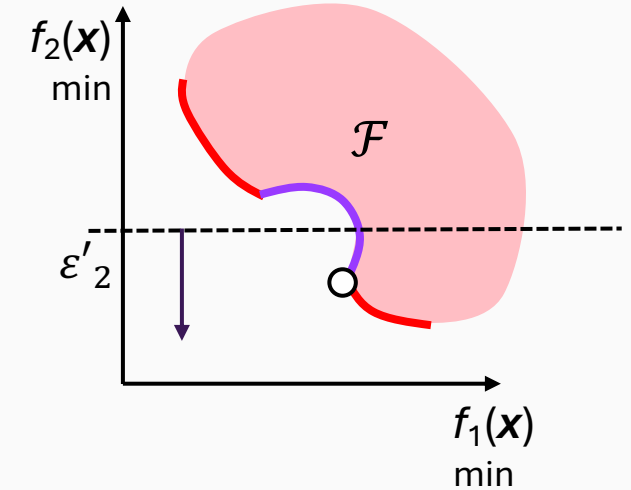
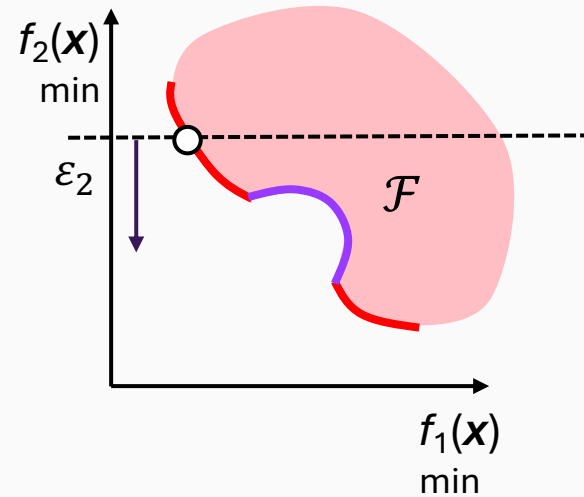


ε -Constraint Method

Haimes et al (1971)

Keep just one of the objective, then treat the rest as constraints.

$$\begin{aligned} & \min_{\mathbf{x} \in \mathcal{X}} f_1(\mathbf{x}) \\ \text{Subject to } & f_2(\mathbf{x}) \leq \varepsilon_2 \\ & \dots \\ & f_m(\mathbf{x}) \leq \varepsilon_m \\ & \mathbf{x} \in \mathcal{X} := \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{g}(\mathbf{x}) \leq 0, \mathbf{h}(\mathbf{x}) = 0\} \end{aligned}$$



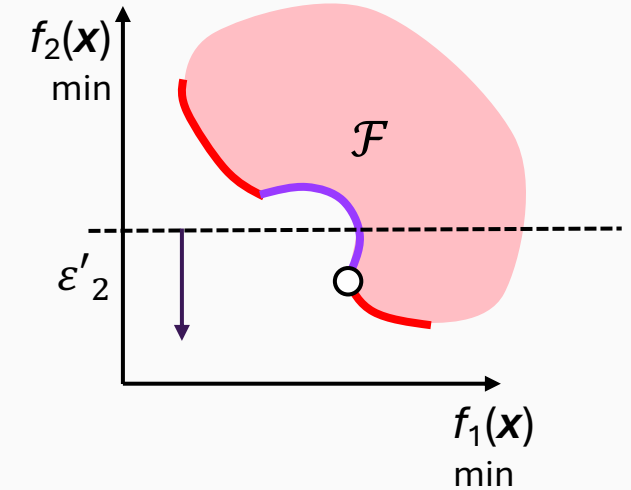
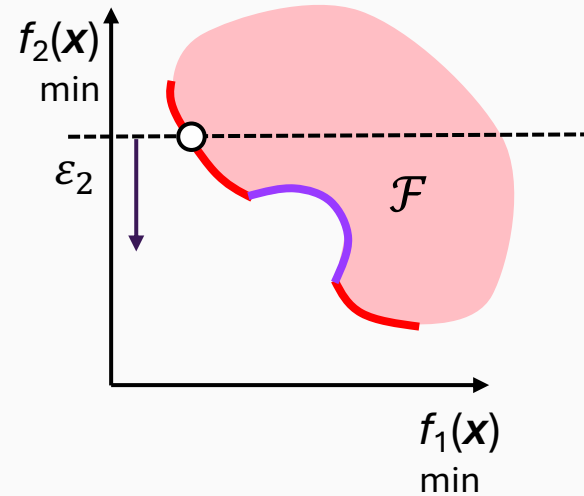
Advantage

- Different Pareto-optimal solutions can be found using different ε values.
- Applicable to either convex or non-convex problems

ϵ -Constraint Method

Keep just one of the objective, then treat the rest as constraints.

$$\begin{aligned} & \min_{\mathbf{x} \in \mathcal{X}} f_1(\mathbf{x}) \\ \text{Subject to } & f_2(\mathbf{x}) \leq \epsilon_2 \\ & \dots \\ & f_m(\mathbf{x}) \leq \epsilon_m \\ & \mathbf{x} \in \mathcal{X} := \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{g}(\mathbf{x}) \leq 0, \mathbf{h}(\mathbf{x}) = 0\} \end{aligned}$$



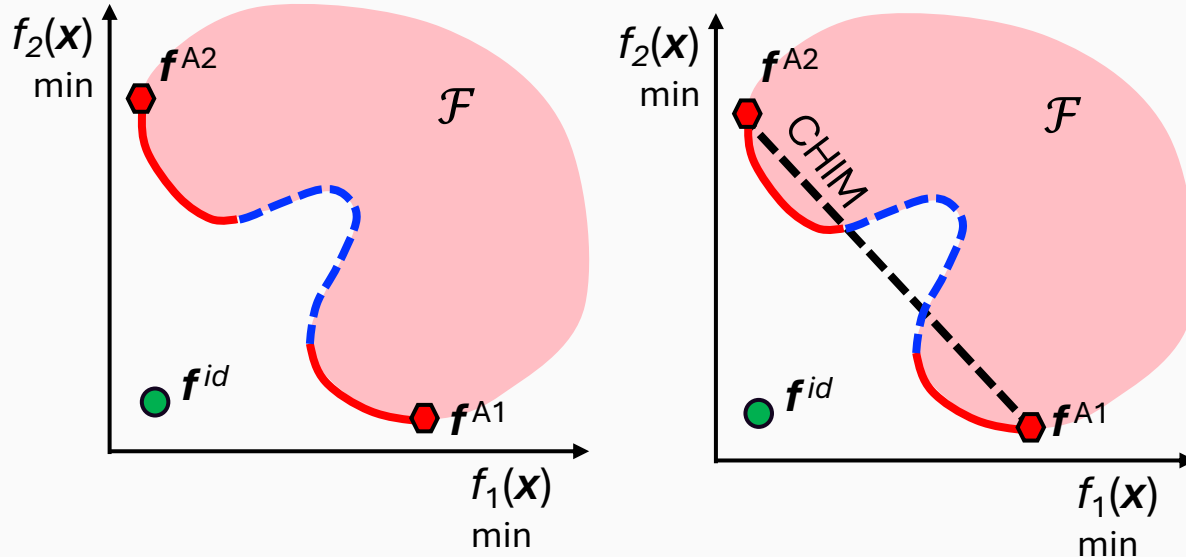
Disadvantage

- The method requires the solution of the SOO problem numerous times, particularly for more than two objectives.
- The additional constraints introduced for the objectives can make the SOO problem difficult to solve.
- Pareto-optimal solutions may not be well distributed.

→ Motivate methods designed more explicitly
to generate representative coverage of the front.

Normal Boundary Intersection (NBI) Method

The original NBI method was proposed by [Das & Dennis \(1998\)](#) to generate uniformly distributed boundary points.



Convex Hull of Individual Minima (CHIM)

$$\left\{ \Phi \beta : \beta \in \mathbb{R}_+^m, \sum_{i=1}^m \beta_i = 1 \right\} \quad \text{where } \Phi \in \mathbb{R}^{m \times m}.$$

For $m=2$

$$\Phi = \begin{bmatrix} f_1^{A1} - f_1^{id} & f_1^{A2} - f_1^{id} \\ f_2^{A1} - f_2^{id} & f_2^{A2} - f_2^{id} \end{bmatrix}$$

Achor point A1
relative to
the ideal point

Achor point A2
relative to
the ideal point

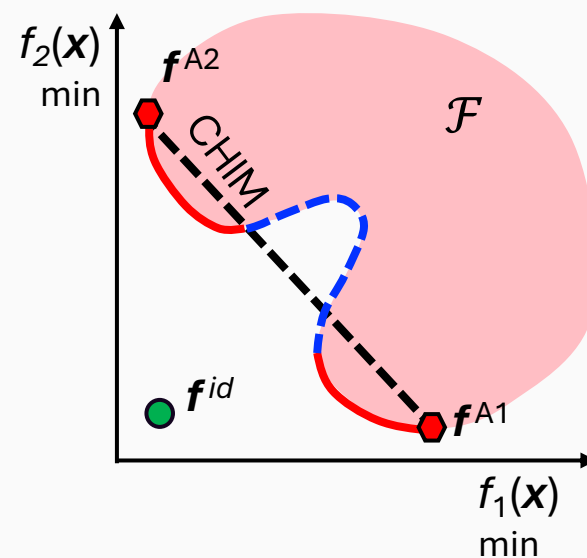
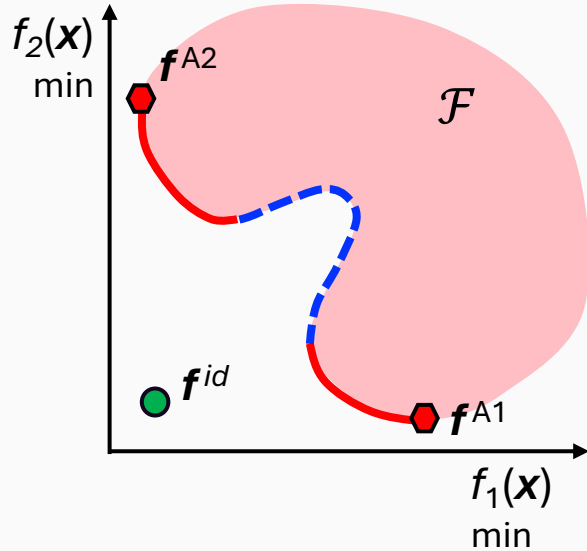
$$\beta = \begin{bmatrix} \beta_1 \\ 1 - \beta_2 \end{bmatrix}$$

Convex combination

$$\begin{aligned} \max_{\mathbf{x} \in \mathcal{X}, t} \quad & t \\ \text{s.t.} \quad & \Phi \beta + t \bar{\mathbf{n}} = \mathbf{f}(\mathbf{x}) - \mathbf{f}^{id}, \\ & t \in \mathbb{R}, \quad \bar{\mathbf{n}} \in \mathbb{R}^m. \end{aligned}$$

Normal Boundary Intersection (NBI) Method

The original NBI method was proposed by [Das & Dennis \(1998\)](#) to generate uniformly distributed boundary points.



Convex Hull of Individual Minima (CHIM)

$$\left\{ \Phi \beta : \beta \in \mathbb{R}_+^m, \sum_{i=1}^m \beta_i = 1 \right\} \quad \text{where } \Phi \in \mathbb{R}^{m \times m}.$$

For $m=2$

$$\Phi = \begin{bmatrix} f_1^{A1} - f_1^{id} & f_1^{A2} - f_1^{id} \\ f_2^{A1} - f_2^{id} & f_2^{A2} - f_2^{id} \end{bmatrix}$$

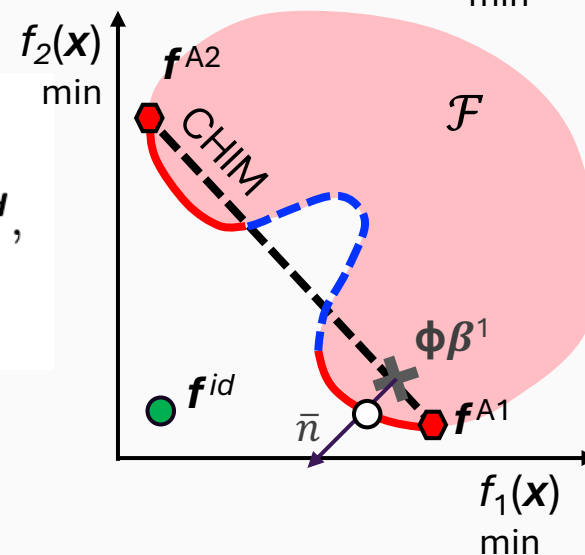
Achor point A1
relative to
the ideal point

Achor point A2
relative to
the ideal point

$$\beta = \begin{bmatrix} \beta_1 \\ 1 - \beta_2 \end{bmatrix}$$

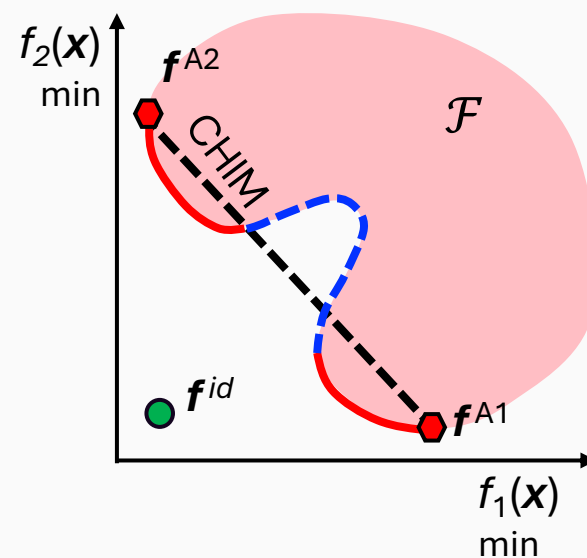
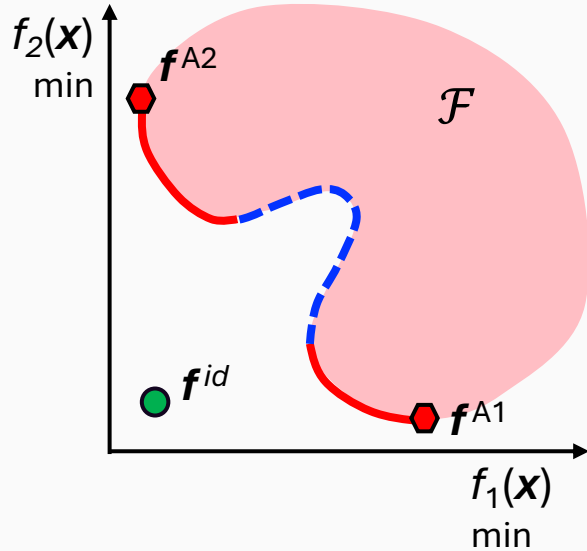
Convex combination

$$\begin{aligned} \max_{x \in \mathcal{X}, t} \quad & t \\ \text{s.t.} \quad & \Phi \beta + t \bar{n} = f(x) - f^{id}, \\ & t \in \mathbb{R}, \quad \bar{n} \in \mathbb{R}^m. \end{aligned}$$



Normal Boundary Intersection (NBI) Method

The original NBI method was proposed by [Das & Dennis \(1998\)](#) to generate uniformly distributed boundary points.



Convex Hull of Individual Minima (CHIM)

$$\left\{ \Phi \beta : \beta \in \mathbb{R}_+^m, \sum_{i=1}^m \beta_i = 1 \right\} \quad \text{where } \Phi \in \mathbb{R}^{m \times m}.$$

For $m=2$

$$\Phi = \begin{bmatrix} f_1^{A1} - f_1^{id} & f_1^{A2} - f_1^{id} \\ f_2^{A1} - f_2^{id} & f_2^{A2} - f_2^{id} \end{bmatrix}$$

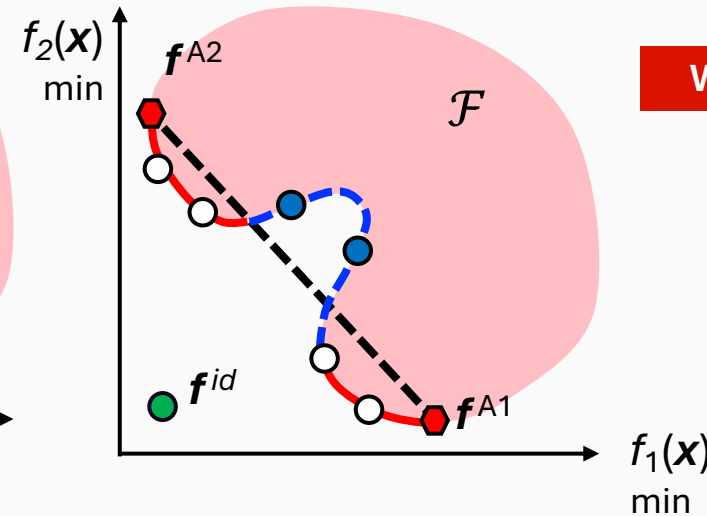
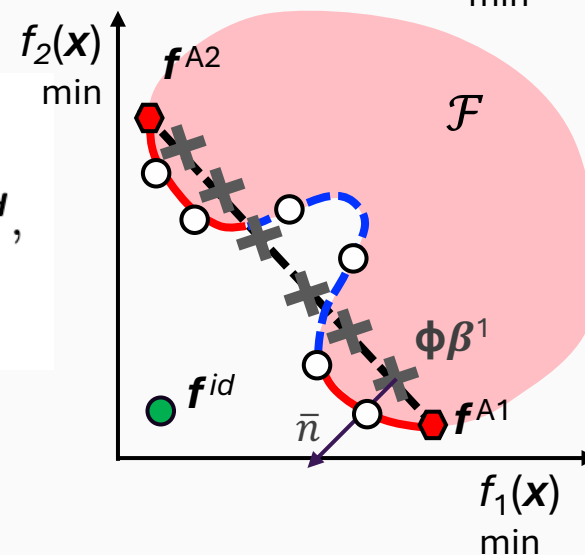
$$\beta = \begin{bmatrix} \beta_1 \\ 1 - \beta_2 \end{bmatrix}$$

Convex combination

Achor point A1
relative to
the ideal point

Achor point A2
relative to
the ideal point

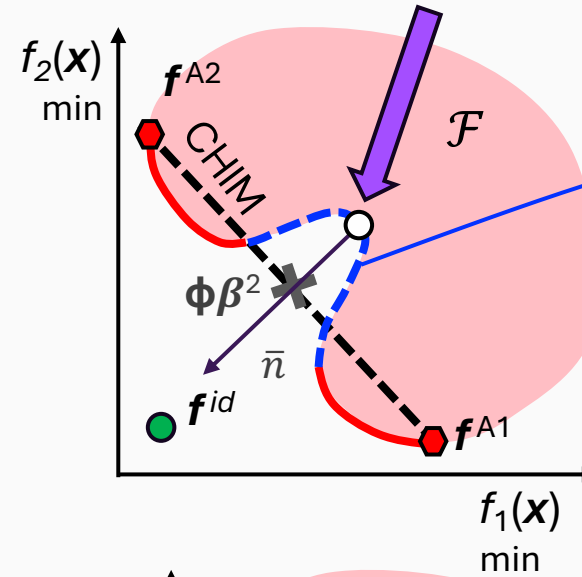
$$\begin{aligned} \max_{x \in \mathcal{X}, t} \quad & t \\ \text{s.t.} \quad & \Phi \beta + t \bar{n} = f(x) - f^{id}, \\ & t \in \mathbb{R}, \quad \bar{n} \in \mathbb{R}^m. \end{aligned}$$



What is the issue here?

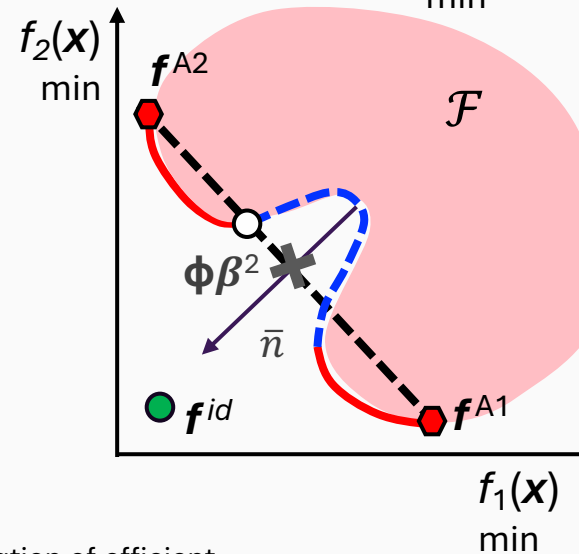
Modified NBI (mNBI) Method

$$\begin{aligned} \max_{\mathbf{x} \in \mathcal{X}, t} \quad & t \\ \text{s.t.} \quad & \Phi\beta + t\bar{\mathbf{n}} = \mathbf{f}(\mathbf{x}) - \mathbf{f}^{id}, \\ & t \in \mathbb{R}, \quad \bar{\mathbf{n}} \in \mathbb{R}^m. \end{aligned}$$



Boundary of feasible region, not Parto points

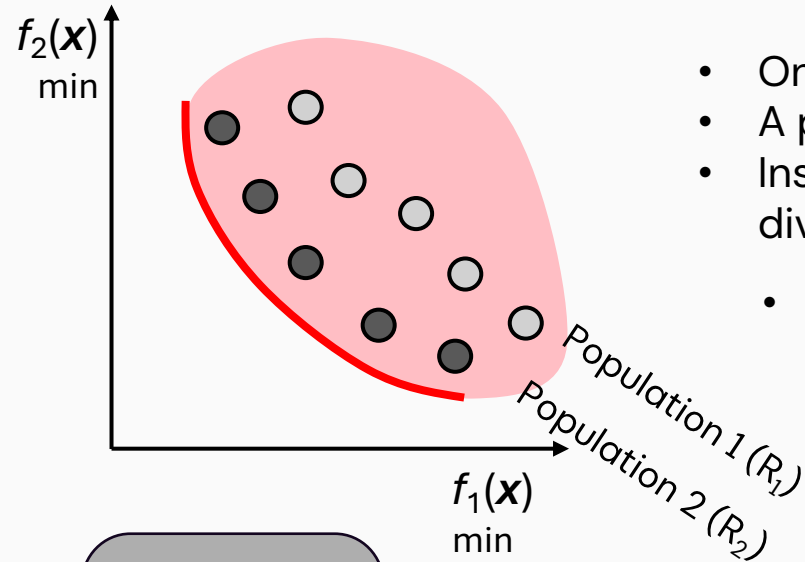
$$\begin{aligned} \max_{\mathbf{x} \in \mathcal{X}, t} \quad & t \\ \text{s.t.} \quad & \Phi\beta + t\bar{\mathbf{n}} \geq \mathbf{f}(\mathbf{x}) - \mathbf{f}^{id}, \\ & t \in \mathbb{R}, \quad \bar{\mathbf{n}} \in \mathbb{R}^m. \end{aligned}$$



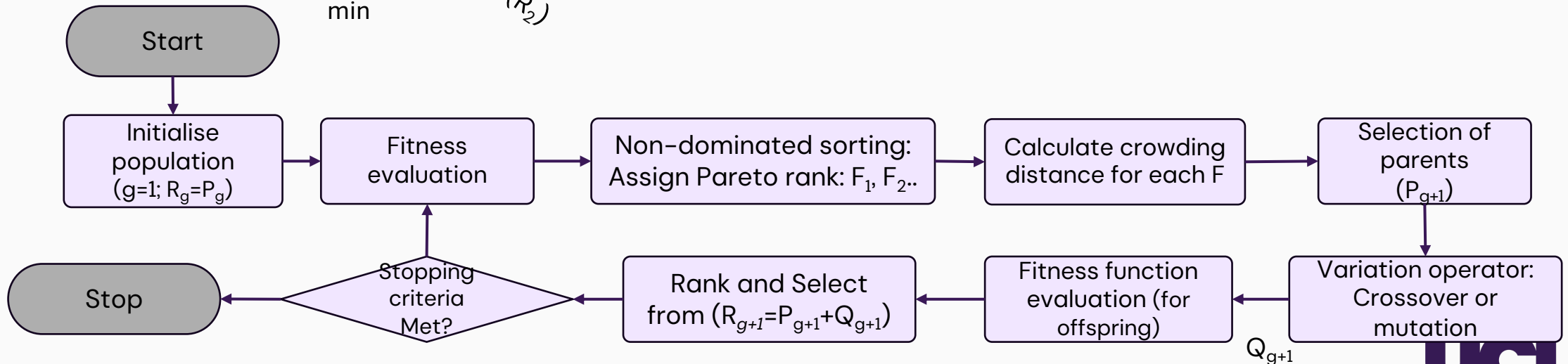
This gives the optimizer freedom to select a feasible objective vector on the preferable side of the normal line, rather than restricting it to the exact intersection.

Evolutionary Algorithm – NSGA-II

Non-scalarization method – seeking a population of trade-offs



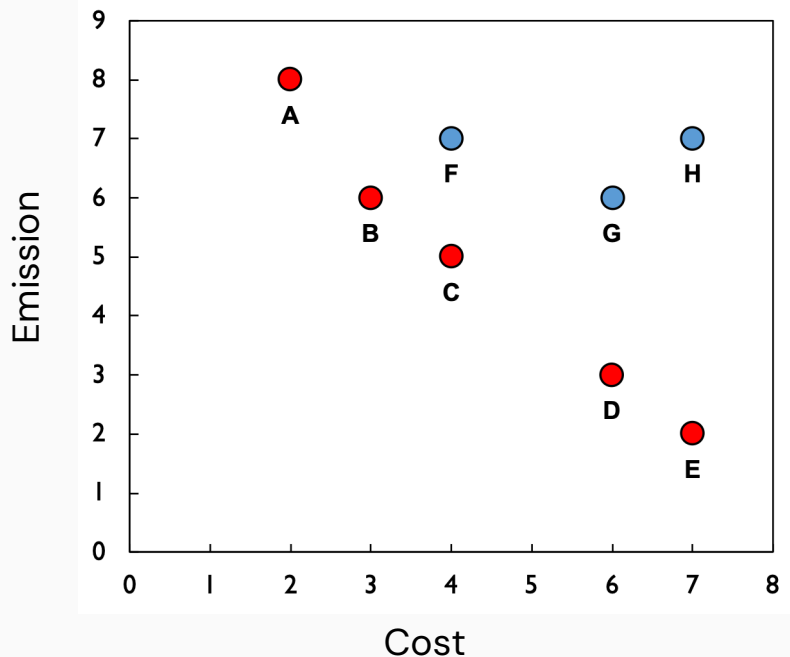
- One of the most popular MOO EAs.
- A population-based algorithm.
- Instead of producing one trade-off, it evolved a population toward a diverse approximation of the Pareto front.
- **Two central idea**
 - **Elite** preserving: to keep the best solutions based on dominance.
 - **Diversity** preserving: to encourages solutions to remain distributed across the front.



Evolutionary Algorithm – NSGA-II

- Example: minimization of **Cost** vs. **CO₂ Emission** (Population size $N=4$)

- **2nd Iteration**



- ✓ Evaluation of function values of $R_g = (P_g + Q_g)$

Candidate	Origin	Cost	Emissions	Pareto rank
A	Parent	2	8	1
B	Parent	3	6	1
C	Offspring	4	5	1
D	Offspring	6	3	1
E	Offspring	7	2	1
F	Parent	4	7	2
G	Offspring	6	6	2
H	Parent	7	7	3

- ✓ **Non-dominated sorting:** Assign Pareto rank: F1, F2..

Candidates are sorted using the concept of Pareto dominance.

First front: $F_1 = \{A, B, C, D, E\}$

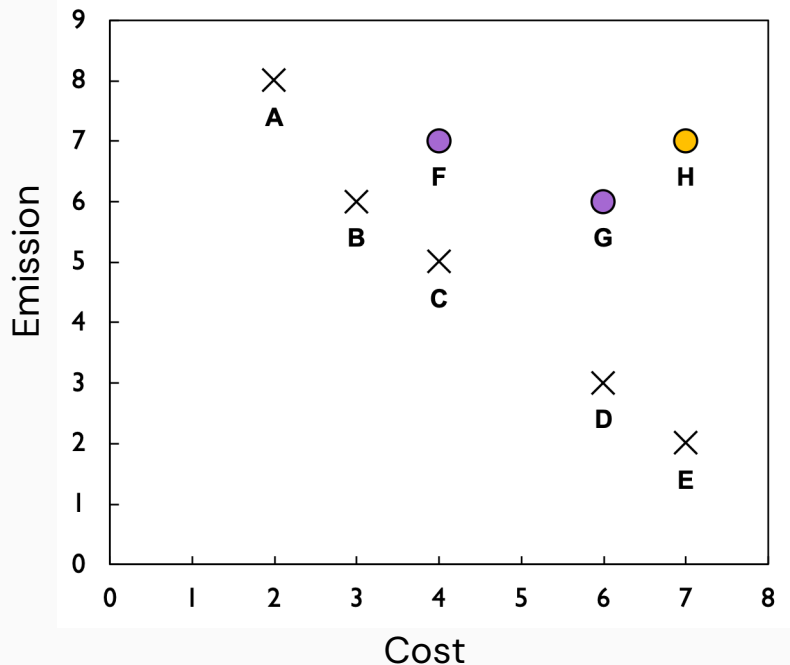
$B = (3, 6)$ dominates $F = (4, 7)$;

$C = (4, 5)$ dominates $G = (6, 6)$.

Evolutionary Algorithm – NSGA-II

- Example: minimization of **Cost** vs. **CO₂ Emission** (Population size $N=4$)

- **2nd Iteration**



- ✓ Evaluation of function values of $R_g = (P_g + Q_a)$

Candidate	Origin	Cost	Emissions	Pareto rank
A	Parent	2	8	1
B	Parent	3	6	1
C	Offspring	4	5	1
D	Offspring	6	3	1
E	Offspring	7	2	1
F	Parent	4	7	2
G	Offspring	6	6	2
H	Parent	7	7	3

- ✓ **Non-dominated sorting:** Assign Pareto rank: F1, F2..

Candidates are sorted using the concept of Pareto dominance.

Second front: $F_2 = \{F, G\}$

Third front: $F_3 = \{H\}$

Note

Rank 1 does not mean that A is better than E .
They are alternative trade-offs.
Rank only counts Pareto layers: lower rank is preferred



Evolutionary Algorithm – NSGA-II

- Example: minimization of **Cost** vs. **CO₂ Emission** (Population size $N=4$)

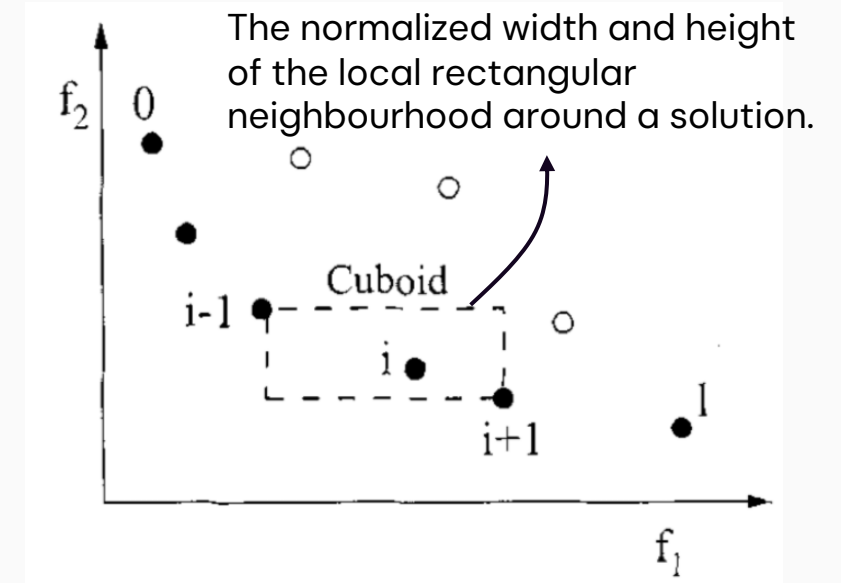
- 2nd Iteration

We have {A, B, C, D, E} in Rank 1 set. We only want to keep 4 parents.

- ✓ **Calculation of crowding distance (d_i)** for diversity
- ✓ It estimates the **amount of empty objective space surrounding each candidate**.
- ✓ When a front does not fit entirely into P_{g+1} , retain its **least crowded** candidates.

$$d_i = \sum_{k=1}^m \frac{f_k(i+1) - f_k(i-1)}{f_k^{\max} - f_k^{\min}}.$$

- ✓ For each objective, candidates are sorted first;
- ✓ For objective k , we measure the distance between two neighbours of solution $(i+1, i-1)$
- ✓ The two boundary candidates are assigned as $d_i = \infty \rightarrow$ ensure that extreme trade-offs are preserved.



Note

Prefer the candidate with larger d_i . This preserves sparsely represented regions and discourages all solutions from clustering on one part of the Pareto front.

Evolutionary Algorithm – NSGA-II

- Example: minimization of **Cost** vs. **CO₂ Emission** (Population size $N=4$)

- 2nd Iteration

- ✓ Calculation of crowding distance (d_i) for diversity

$$d_i = \sum_{k=1}^m \frac{f_k(i+1) - f_k(i-1)}{f_k^{\max} - f_k^{\min}}.$$

$$d_A = d_E = \infty$$

$$d_B = \frac{f_1(C) - f_1(A)}{5} + \frac{f_2(C) - f_2(A)}{5} = 0.9,$$

$$d_C = 1.1, \quad d_D = 1.1$$

$$P_{g+1} = \{A, C, D, E\}, \quad |P_{g+1}| = 4$$

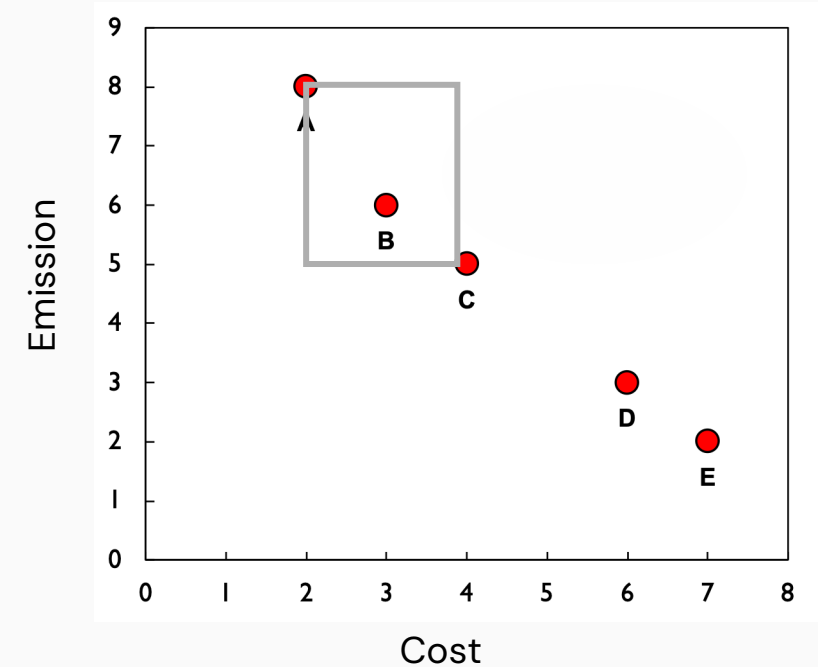
- ✓ Selection of parents & off-spring generation

Parents pair (A, C) and (E, D)

design space x

$$A = [1, 1, 0, 0, 1, 0], \quad C = [1, 0, 1, 1, 0, 0],$$

$$E = [0, 0, 1, 1, 1, 1], \quad D = [0, 1, 1, 1, 0, 1],$$



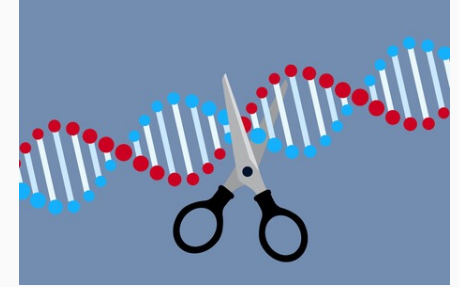
Evolutionary Algorithm – NSGA-II

- Example: minimization of **Cost** vs. **CO₂ Emission** (Population size $N=4$)

- 2nd Iteration

- ✓ Selection of parents & off-spring generation

Parents pair (A, C) and (E, D)



Cross-over example

$A = [1, 1, 0 \mid 0, 1, 0]$, $C = [1, 0, 1 \mid 1, 0, 0]$,
 $U = [1, 1, 0 \mid 1, 0, 0]$, $V = [1, 0, 1 \mid 0, 1, 0]$,
 $E = [0, 0, 1 \mid 1, 1, 1]$, $D = [0, 1, 1 \mid 1, 0, 1]$,
 $W = [0, 0, 1 \mid 1, 0, 1]$, $X = [0, 1, 1 \mid 1, 1, 1]$.

Mutation example

$A = [1, 1, 0, 0, 1, 0] \rightarrow Z = [1, 1, 0, 1, 1, 0]$.

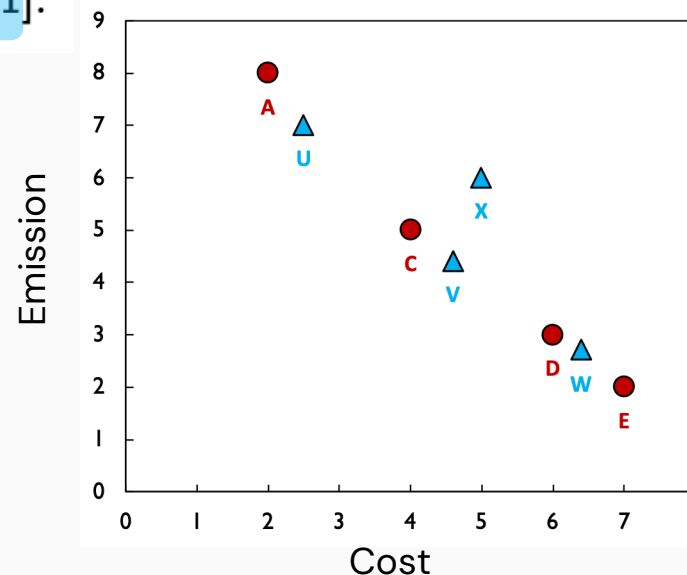
Flip 4th element of A



$Q_{t+1} = \{U, V, W, X\}$

- ✓ Function evaluation and selection (repeat $\rightarrow$ sorting & preserve)

$R_{t+1} = P_{t+1} \cup Q_{t+1} = \{A, C, D, E, U, V, W, X\}$



- **Crossover**: recombines information already present in the selected parents,
- **Mutation**: introduces new variation.

Evolutionary Algorithm – NSGA-II

- **Pros**

- Produces a set of trade-offs in one run → suits a posteriori decision-making.
- Works with black-box and non-smooth problems. No gradients, convexity, or differentiability are required.
- Can handle discrete and mixed variables. The representation and mutation/crossover operators can be designed for binary, integer, continuous, molecular, or flowsheet decisions.

- **Cons**

- **Many objective evaluations.** A population of N over G generations needs roughly $N \times G$ expensive offspring evaluations, plus the initial population.
 - ✓ This is a major issue for experiments, CFD, molecular simulation, or quantum calculations.
- **No exact-optimality guarantee.** The result is an approximate Pareto front.
- Performance depends on encoding and operators. Poor mutation/crossover operators can create infeasible or uninteresting designs.
- **Parameter sensitivity.** Population size, mutation rate, crossover rate, termination rule, and constraint strategy matter.

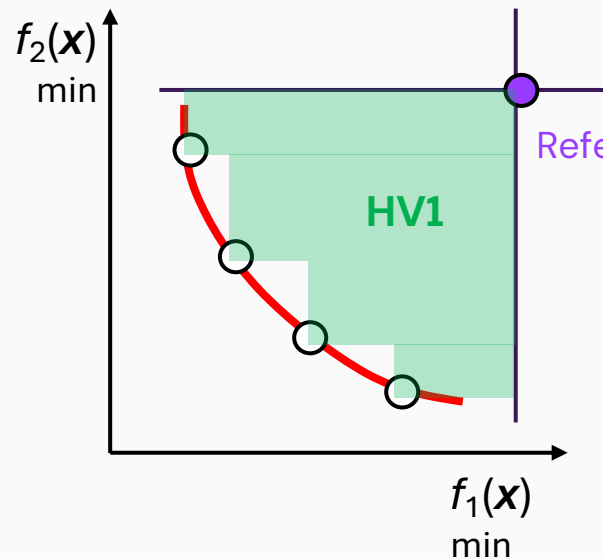
Performance Metrics

Evaluating an approximated Pareto front requires measuring two distinct qualities:
convergence and **diversity**

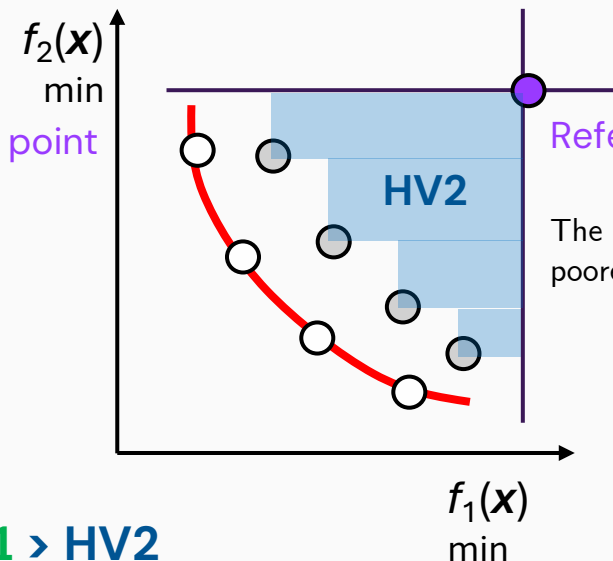
- **Convergence:** how close the points are to the true optimal front
- **Diversity:** how evenly distributed the points are across the objective space

○ Hypervolume (HV)

It calculates the volume (or area in 2D) of the objective space that is dominated by the approximation set, bounded by a predefined, worst-case reference point.



HV1 > HV2



The solutions are farther from the front and provide poorer coverage, giving the smaller.

Capture both convergence and diversity

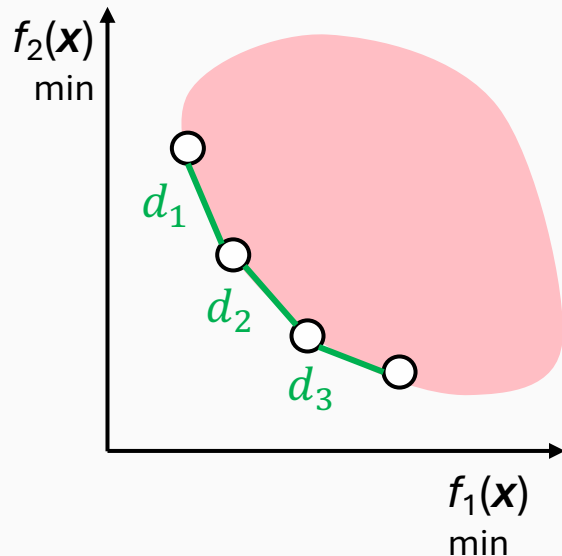
Performance Metrics

Hypervolume combines convergence and diversity into one number, but it **does not tell us** which of the two caused the difference.

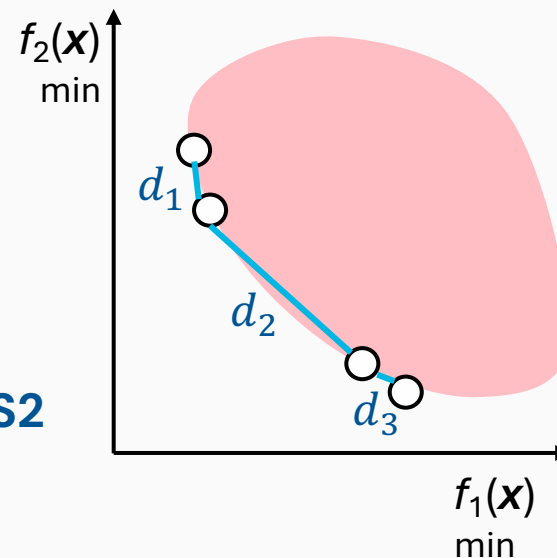
- **Spacing (S)** It measures the *variance* of the distances between neighbouring vectors in the approximated Pareto front → Diversity

$$S = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (\bar{d} - d_i)^2}$$

d_i is the distance from solution i to its nearest neighbour, and $\bar{d}$ is the mean of all d_i .



$S_1 < S_2$



A score of 0 means all points are perfectly, evenly spaced.

Also others..

- **More on MOO algorithms**

Ehrgott, M. (2005) Multicriteria optimization. 2nd edn. Berlin and Heidelberg: Springer. Available at: <https://doi.org/10.1007/3-540-27659-9>.

Deb, K. (2001) Multi-objective optimization using evolutionary algorithms. Chichester: John Wiley & Sons.

Antunes, C.H., Alves, M.J. and Clímaco, J. (2016) Multiobjective linear and integer programming. Cham: Springer. doi: 10.1007/978-3-319-28746-1.

Also others..

Sandwich Algorithm (SD) for the guided choice of w of weighted sum subproblem

- Approximate the (convex) Pareto front with **as few optimisation runs as possible**.
- Original sandwich algorithm (Solanki et al., 1993 & Rennen et al., 2011) uses a weighted sum subproblem.
- It **iteratively constructs a convex hull (inner approximation) and outer approximation** between which the Pareto front is sandwiched.

$$\min_{\mathbf{x} \in \mathcal{X}} \sum_{i=1}^m w_i \tilde{f}_i(\mathbf{x}), \quad w_i \geq 0, \quad \sum_i w_i = 1$$

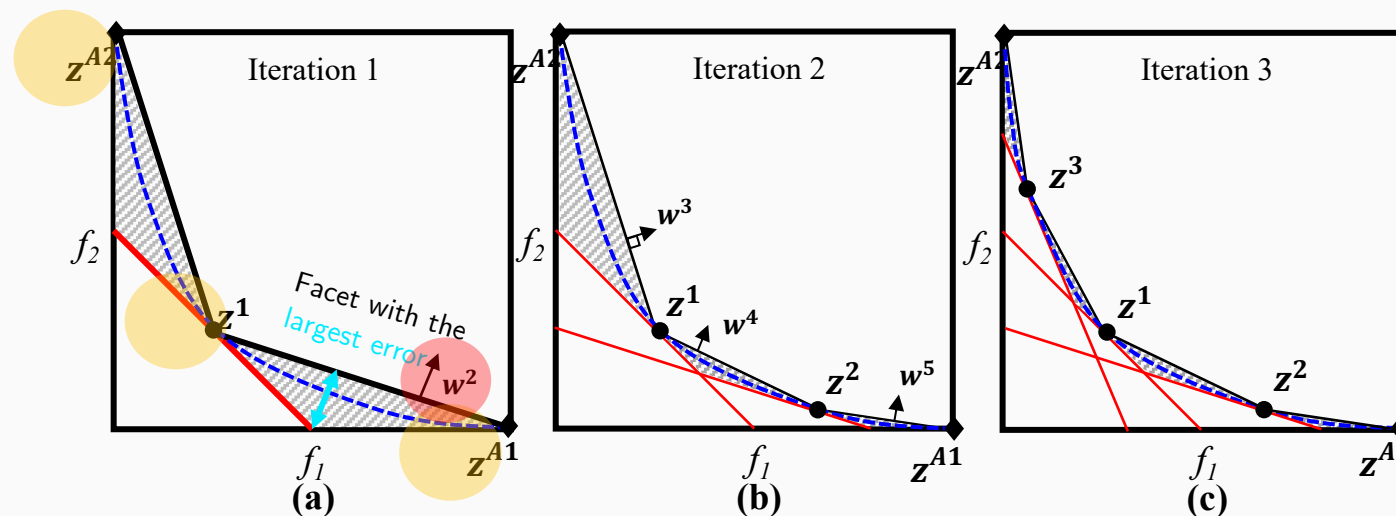


Figure is based on Figure 1 in Lee et al. (2020)

Also others..

SDNBI: reliable solution of general nonconvex and discrete BOO problem

Lee, Y.S., Jackson, G., Galindo, A. and Adjiman, C.S., 2024. A deterministic optimization algorithm for nonconvex and combinatorial bi-objective programming. arXiv preprint [arXiv:2410.17190](https://arxiv.org/abs/2410.17190).

Application of MOO in Molecular and Process design

Solvent design for chemical absorption of CO₂ capture

$$\min_n C_P, RED$$

$$\text{s.t. } g(n) \leq 0$$

$$h(n) = 0$$

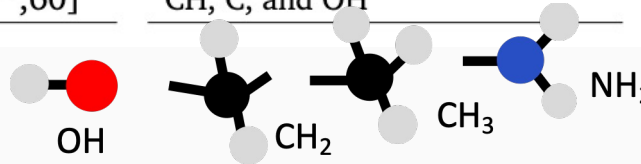
$$Cn \leq d$$

Property constraints

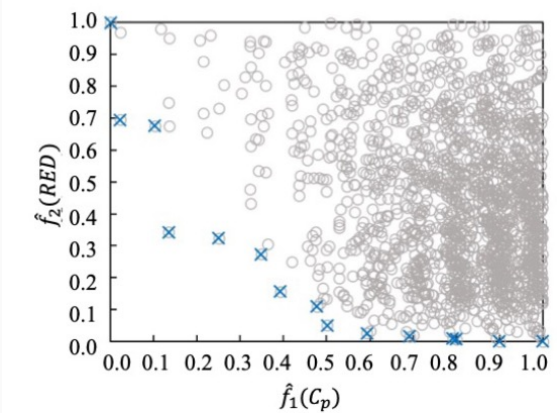
Physical properties, $g(n)$	Bounds
ρ (g/cm ³) at 25°C and 1 atm	[0.6, 1.5]
RED	[10 ⁻⁵ , 6.5]
T_b (K) at 1 atm	[393, 550]
T_m (K) at 1 atm	[273, 313]
σ (dyn/cm) at 25°C	[25, 60]
μ (cP) at 40°C	[10 ⁻⁵ , 60]

Molecular design space

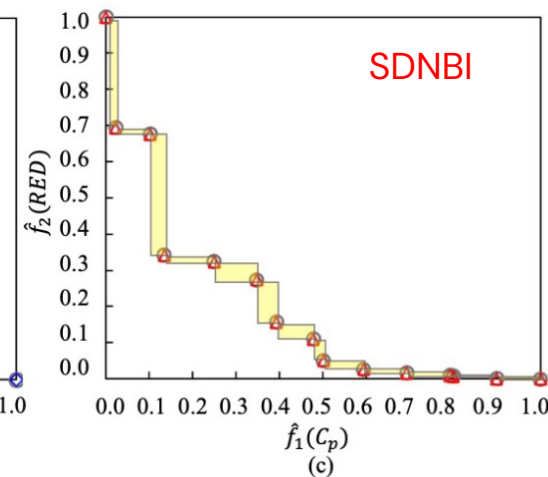
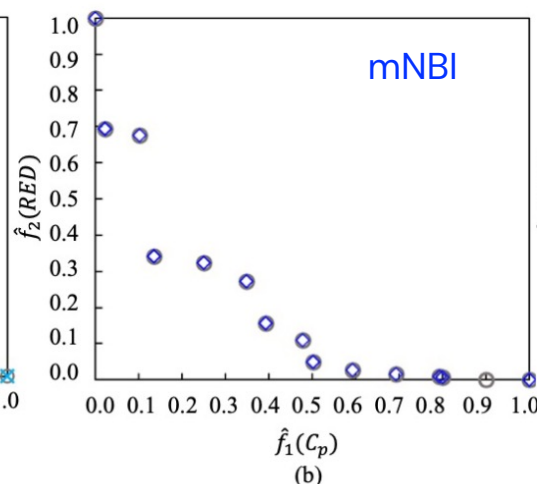
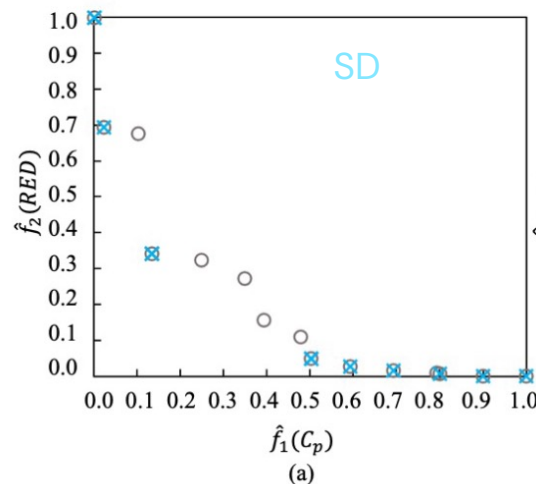
Functional groups	Bounds
CH ₂ N, CH ₃ N,	$n_{tot} = 13$ $n_{tot,A} = 5$
CHNH, CH ₂ NH	
CH ₃ NH, CNH ₂ ,	
CHNH ₂ , CH ₂ NH ₂ ,	
CH ₃ , CH ₂ ,	
CH, C, and OH	



True-Pareto front



Results: SD, mNBI, and SDNBI



MOO in AI/ML domain

In the AI era, MOO appears in three different roles

MOO for AI

Choose a model trading accuracy against latency, energy, size, fairness, or robustness.

MOO in AI

Train one shared model for tasks with competing losses and gradients.

AI for MOO

Use learned surrogate models to reduce expensive simulations or experiments.

e.g., Multi-objective Bayesian optimization for experimental design

The central idea survives unchanged

We still generate non-dominated alternatives, assess their quality, and choose using explicit preferences.

AI changes the search space and the cost of evaluation.

Thank you!
Q&A